

1. Path planning when both lane detected



이전에 배웠던 것들을 총 동원해서 Global Plane 위에서 경로를 따라 움직이는 자동차를 구현해 봅시다.

실제 자동차에서의 환경을 모사하기 위해 주행중인 자차에서 Global plane 위의 point 들을 관측했을 때 어떻게 되는지, 관측한 값을 차선으로 어떻게 나타낼 지, 나타낸 양 차선을 어떻게 활용하여 자동차에게 Path 를 줄지 고민하고 코드로 구현해 보세요

참고로, 자동차 거동 모델과 제어기(Pure pursuit)은 여러분들의 수고를 덜기 위해 제공해드렸습니다!

2. Path planning when either lane detected

자차의 위치와 차선의 위치를 인식하고, 차선의 위치를 따라가도록 제어한다.

양 차선 모두 인식되었을 때는 너무 쉬웠나요?

이번에도 그렇게 어렵지 않습니다. 한 쪽 차선만 인식되었을 때 자동차에게 Path 를 생성하고, 제어해 보세요

3. Path planning by leading vehicle



이번에는 양 차선 모두 없이 내 앞에서 주행중인 leading vehicle을 따라가도록 Path planning 해보세요!

4. Dijkstra algorithm



이번엔 기본적인 Searching based planning 입니다.
배웠던 Dijkstra 를 구현해 보세요. Action set은 각각 상, 하, 좌, 우, 각 대각선으로 총 8개의 Action을 사용해 보세요.

5. A star algorithm



기존에 구현한 Dijkstra 에 Heuristic을 추가하여 A star algorithm을 구현해 보세요. Weight 를 바꿔가며 Weighted A star 로도 활용해 보세요

6. Hybrid a star algorithm



실제 자동차의 거동을 고려할 수 있도록 Hybrid a star algorithm을 구현해 보세요.

Action은 후진 없이 직진/좌/우회전 으로만 구성해서 문제를 풀어보세요.

Hint : continuous한 space에 있으니 정확하게 동일한 위치에 노드들이 위치할 수 없습니다. 적당히 가까우면 동일 노드라고 치죠!

7. RRT

초기 위치 목표 위치

RRT 로 목적지까지 빠르게 도착하는 Algorithm을 만들어 보세요.

8. RRT star



조금 더 Optimal 에 가까운 Path 를 뽑기 위해 RRT star를 사용해 보세요.

Reference code는 networkx 패키지를 사용했으나, 영 마음에 맞지 않는다면 사용하시 않으셔도 무관합니다.

매번 바뀌는 지도가 맘에 들지 않는다면, map 파일의 주석 처리된 random seed를 고정 후 사용하셔도 됩니다.

9. Dubins path



두 점을 Dubins path 로 연결하는 코드를 작성해보세요(Reference Code 제공)
꼭 손으로 코드를 작성해서 구현해보길 바랍니다.

[illegible]

꼭 손으로 코드를 작성해서 구현해보길 바랍니다.